

In [22]: *# Univariate, Bivariate and Multivariate Analysis*

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
import seaborn as sns

# Load iris dataset
iris = load_iris()

# Create dataframe
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Add target column
df['species'] = iris.target

# -----
# 1. Univariate Analysis
# -----

plt.hist(df['sepal length (cm)'], bins=10, edgecolor='black')
plt.title("Univariate Analysis")
plt.xlabel("Sepal Length")
plt.ylabel("Count")
plt.show()

# -----
# 2. Bivariate Analysis
# -----

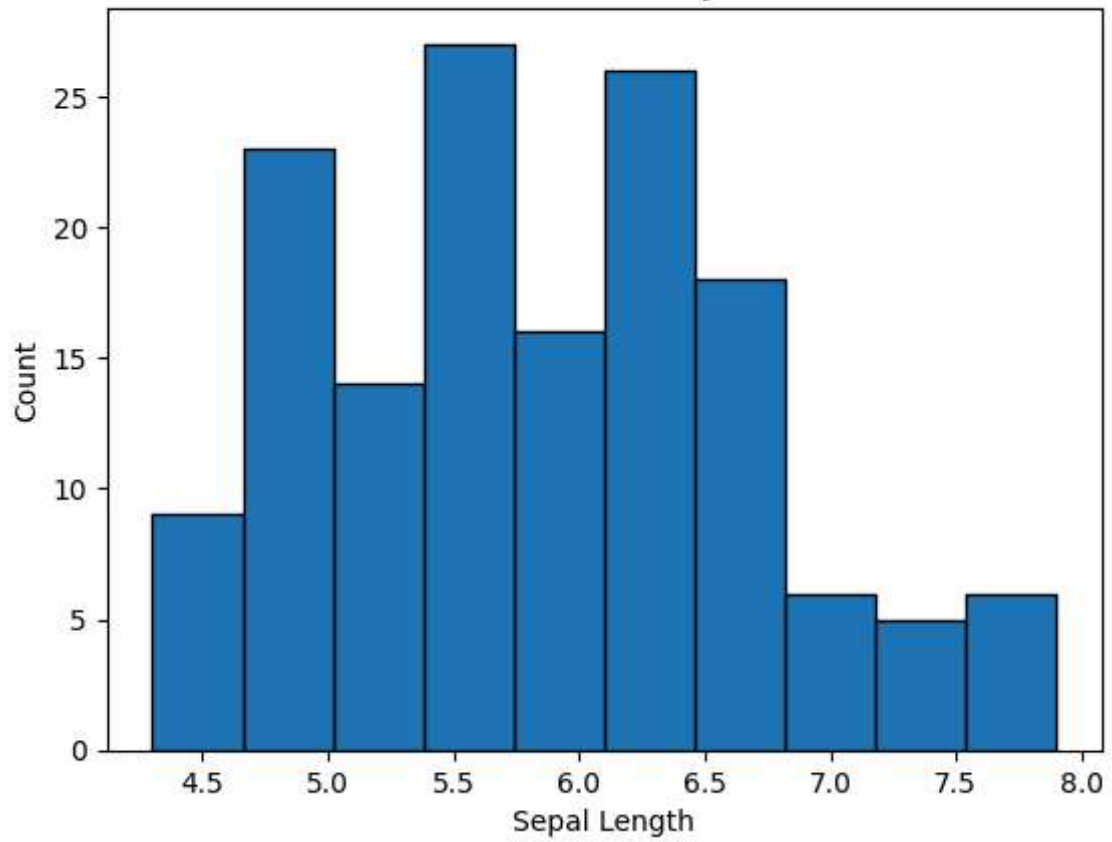
plt.scatter(df['sepal length (cm)'],
            df['petal length (cm)'])

plt.title("Bivariate Analysis")
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")
plt.show()

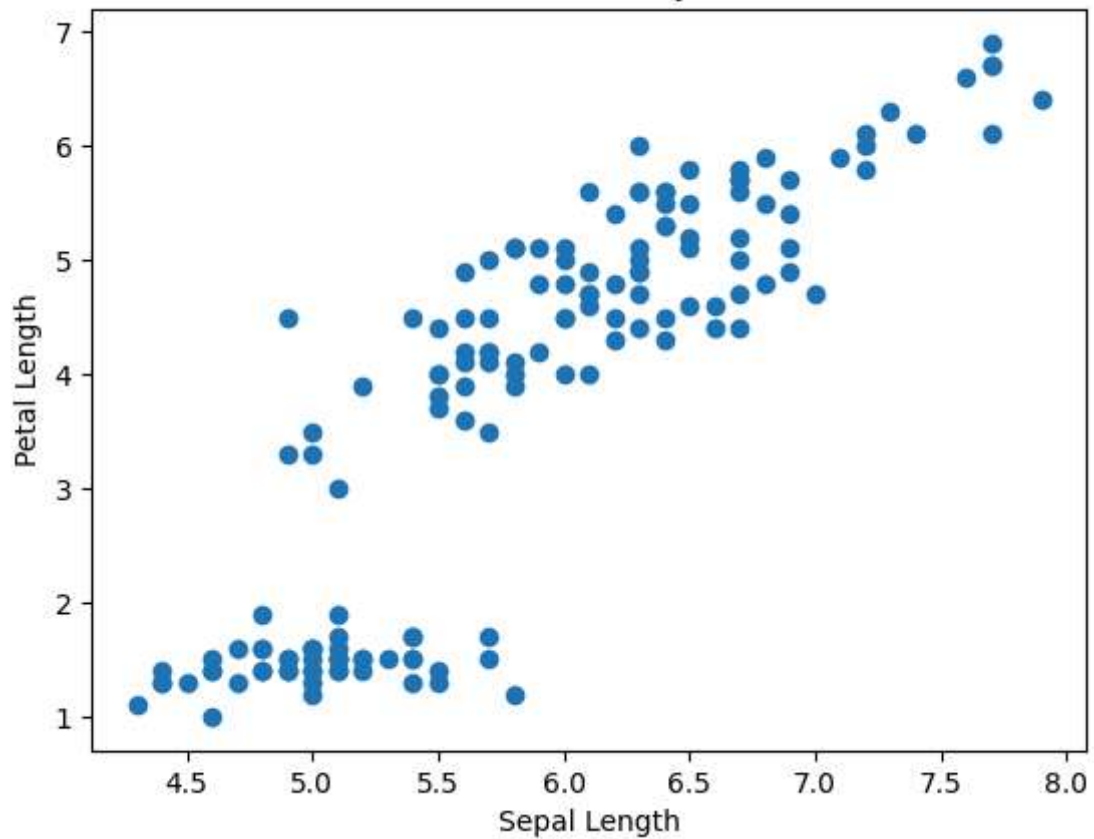
# -----
# 3. Multivariate Analysis
# -----

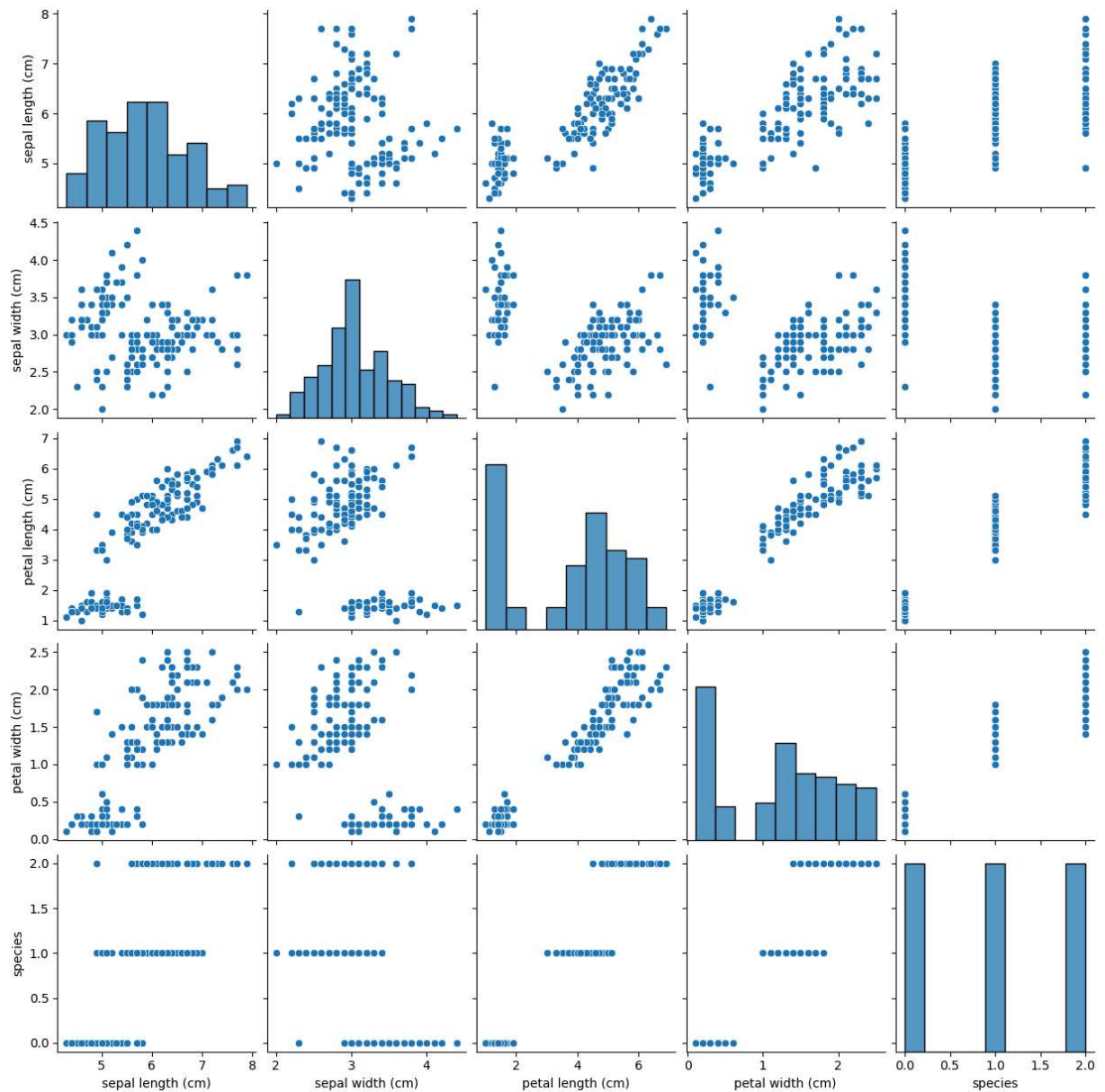
sns.pairplot(df)
plt.show()
```

Univariate Analysis



Bivariate Analysis





In [23]: *# Simple Linear Regression using Least Square Method*

```
import numpy as np
import matplotlib.pyplot as plt

# Input data
x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 5, 4, 5])

# Number of values
n = len(x)

# Calculate slope (m)
m = ((n * np.sum(x * y)) - (np.sum(x) * np.sum(y))) / \
    ((n * np.sum(x * x)) - (np.sum(x) ** 2))

# Calculate intercept (c)
c = (np.sum(y) - m * np.sum(x)) / n
```

```

print("Slope (m):", m)
print("Intercept (c):", c)

# Predicted values
y_pred = m * x + c

# Plot original points
plt.scatter(x, y)

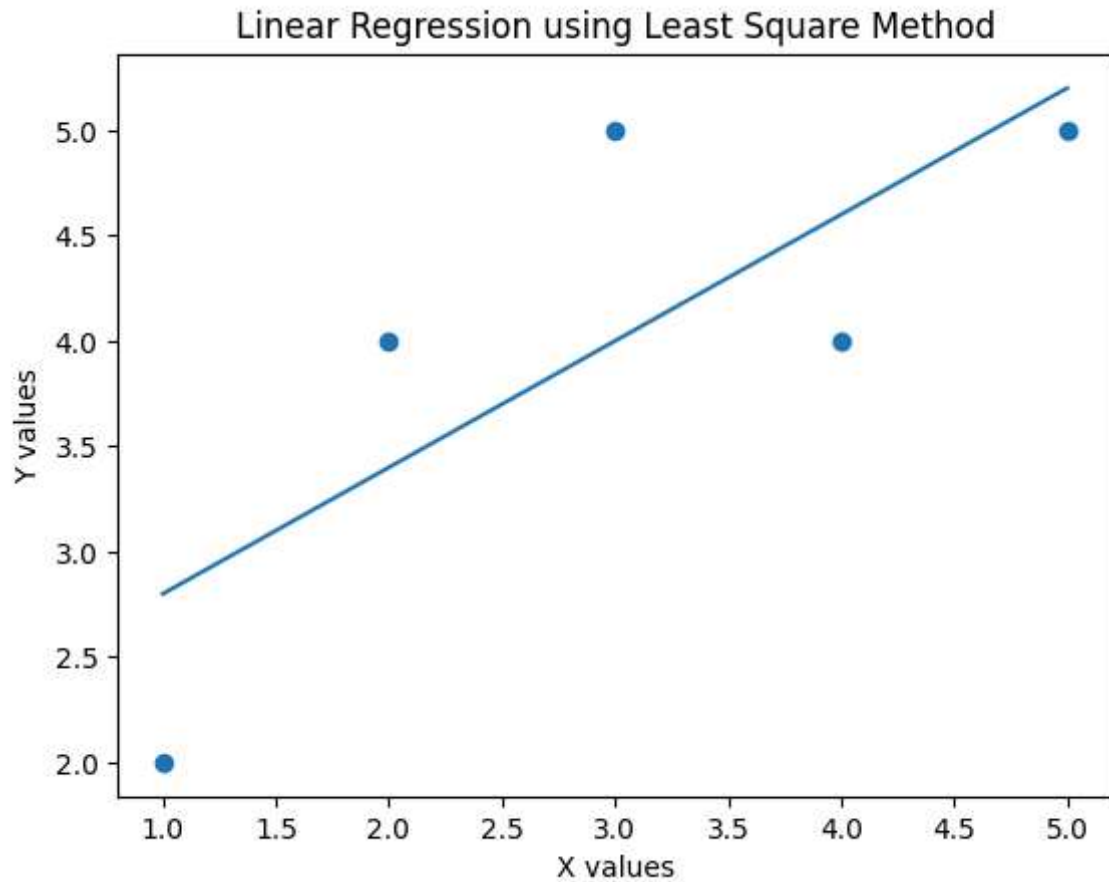
# Plot regression line
plt.plot(x, y_pred)

# Labels
plt.title("Linear Regression using Least Square Method")
plt.xlabel("X values")
plt.ylabel("Y values")

plt.show()

```

Slope (m): 0.6
Intercept (c): 2.2



```

In [24]: # Logistic Regression using Iris Dataset

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression

```

```

from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix

# Load dataset
iris = load_iris()

# Create dataframe
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Input features
X = df[['sepal length (cm)', 'petal length (cm)']]

# Target values
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create model
model = LogisticRegression()

# Train model
model.fit(X_train, y_train)

# Predict values
y_pred = model.predict(X_test)

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
print(cm)

# Scatter plot
plt.scatter(X_test['sepal length (cm)'],
            X_test['petal length (cm)'],
            c=y_pred)

plt.title("Logistic Regression Classification")
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")

plt.show()

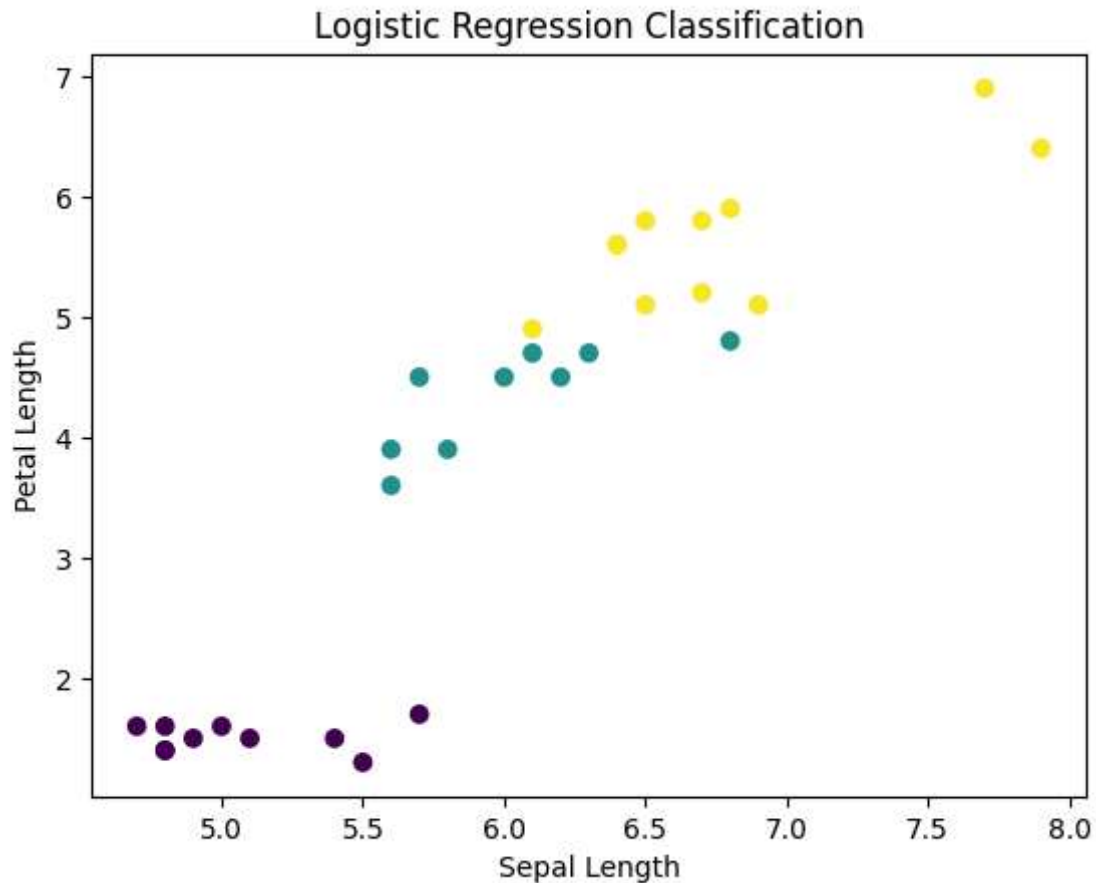
```

Confusion Matrix:

```

[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

```



```
In [25]: # Single Layer Perceptron using Iris Dataset

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()

# Create dataframe
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Input and output
X = df[['sepal length (cm)', 'petal length (cm)']]
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create perceptron model
model = Perceptron()
```

```

# Train model
model.fit(X_train, y_train)

# Predict output
y_pred = model.predict(X_test)

# Accuracy
acc = accuracy_score(y_test, y_pred)

print("Accuracy:", acc)

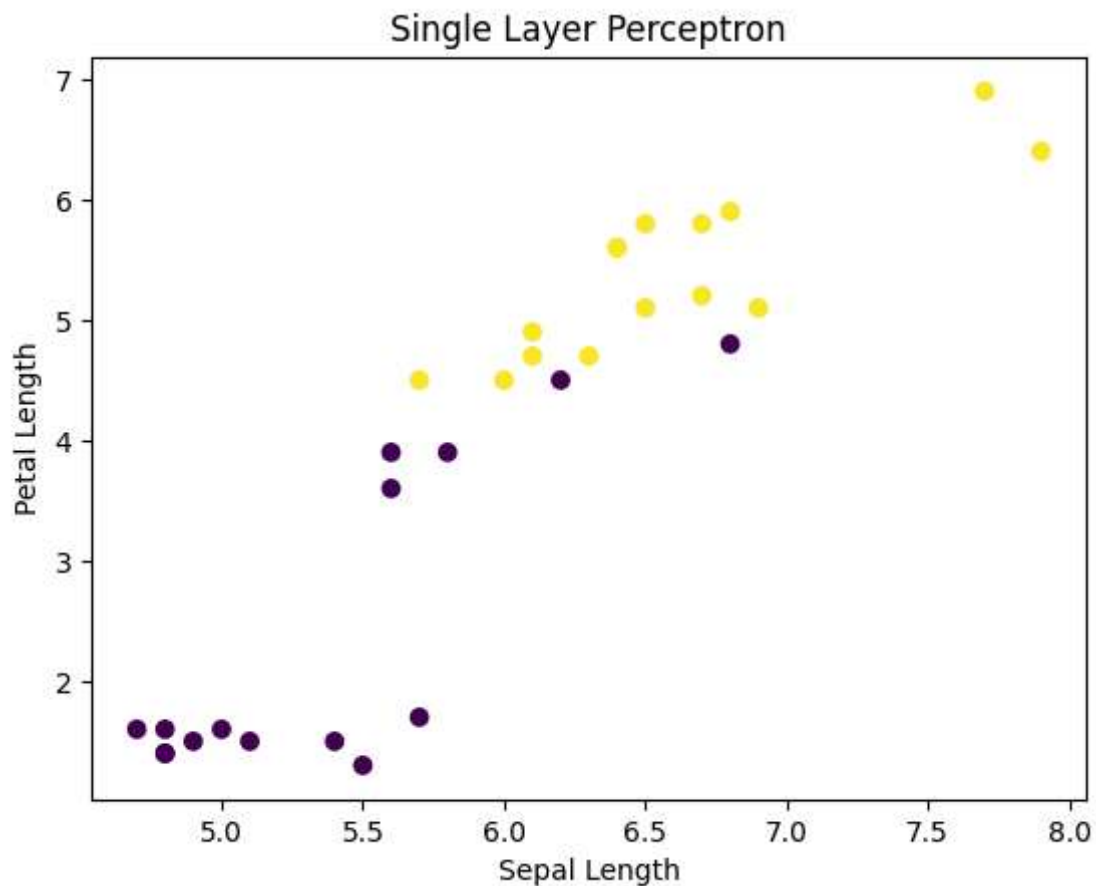
# Plot
plt.scatter(X_test['sepal length (cm)'],
            X_test['petal length (cm)'],
            c=y_pred)

plt.title("Single Layer Perceptron")
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")

plt.show()

```

Accuracy: 0.7



In [26]: # Multi Layer Perceptron using Iris Dataset

```

import pandas as pd
import matplotlib.pyplot as plt

```

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()

# Create dataframe
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Input and output
X = df[['sepal length (cm)', 'petal length (cm)']]
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create MLP model
model = MLPClassifier(
    hidden_layer_sizes=(5,),
    max_iter=1000,
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Predict output
y_pred = model.predict(X_test)

# Accuracy
acc = accuracy_score(y_test, y_pred)

print("Accuracy:", acc)

# Plot
plt.scatter(X_test['sepal length (cm)'],
            X_test['petal length (cm)'],
            c=y_pred)

plt.title("Multi Layer Perceptron")
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")

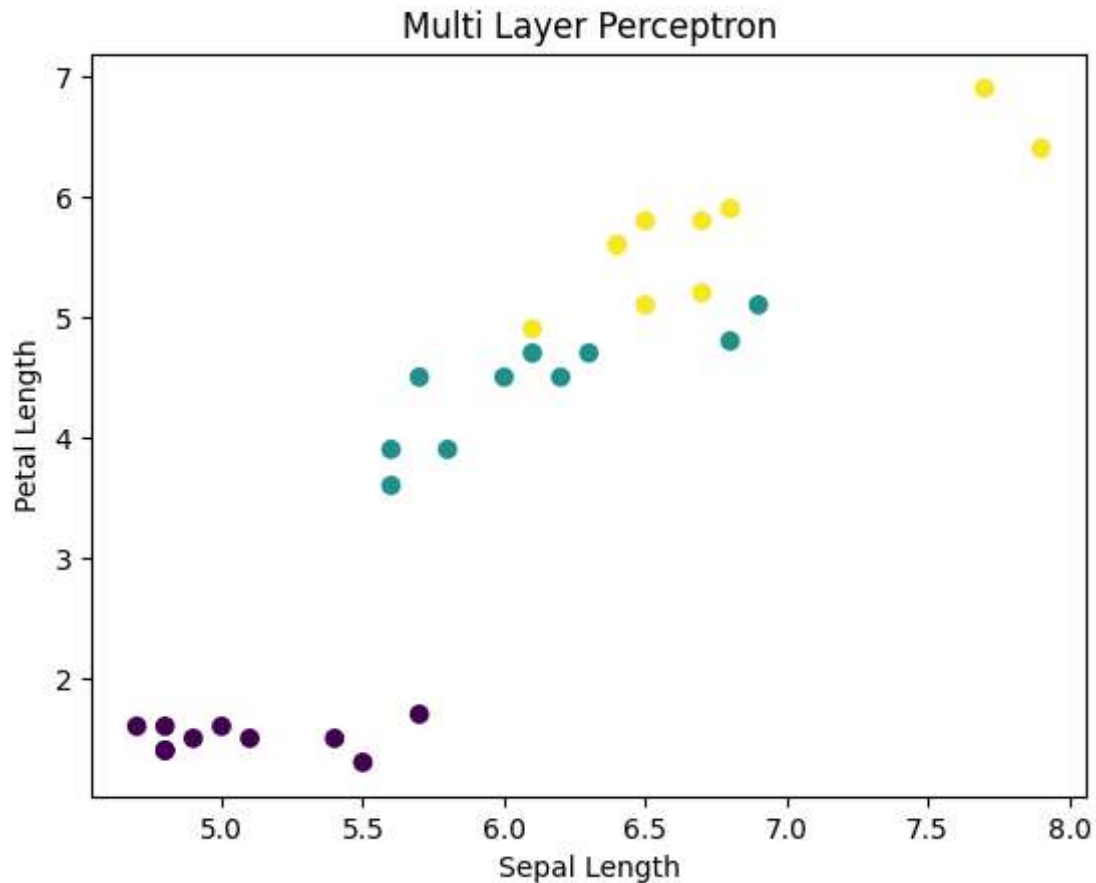
plt.show()

```

C:\Users\Janani\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\neural_network_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.

warnings.warn(

Accuracy: 0.9666666666666667



```
In [27]: # Face Recognition using SVM

import matplotlib.pyplot as plt

from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Load face dataset
faces = fetch_olivetti_faces()

# Input data
X = faces.data

# Target Labels
y = faces.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create SVM model
model = SVC(kernel='linear')

# Train model
model.fit(X_train, y_train)
```

```

# Predict output
y_pred = model.predict(X_test)

# Accuracy
acc = accuracy_score(y_test, y_pred)

print("Accuracy:", acc)

# Display one test image
plt.imshow(X_test[0].reshape(64, 64), cmap='gray')

# Predicted Label
plt.title("Predicted Face: " + str(y_pred[0]))

plt.axis('off')
plt.show()

```

Accuracy: 0.975

Predicted Face: 20



```

In [28]: # Decision Tree using Iris Dataset

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset

```

```

iris = load_iris()

# Create dataframe
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Input and output
X = df
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create Decision Tree model
model = DecisionTreeClassifier()

# Train model
model.fit(X_train, y_train)

# Predict output
y_pred = model.predict(X_test)

# Accuracy
acc = accuracy_score(y_test, y_pred)

print("Accuracy:", acc)

# Plot decision tree
plt.figure(figsize=(10, 6))

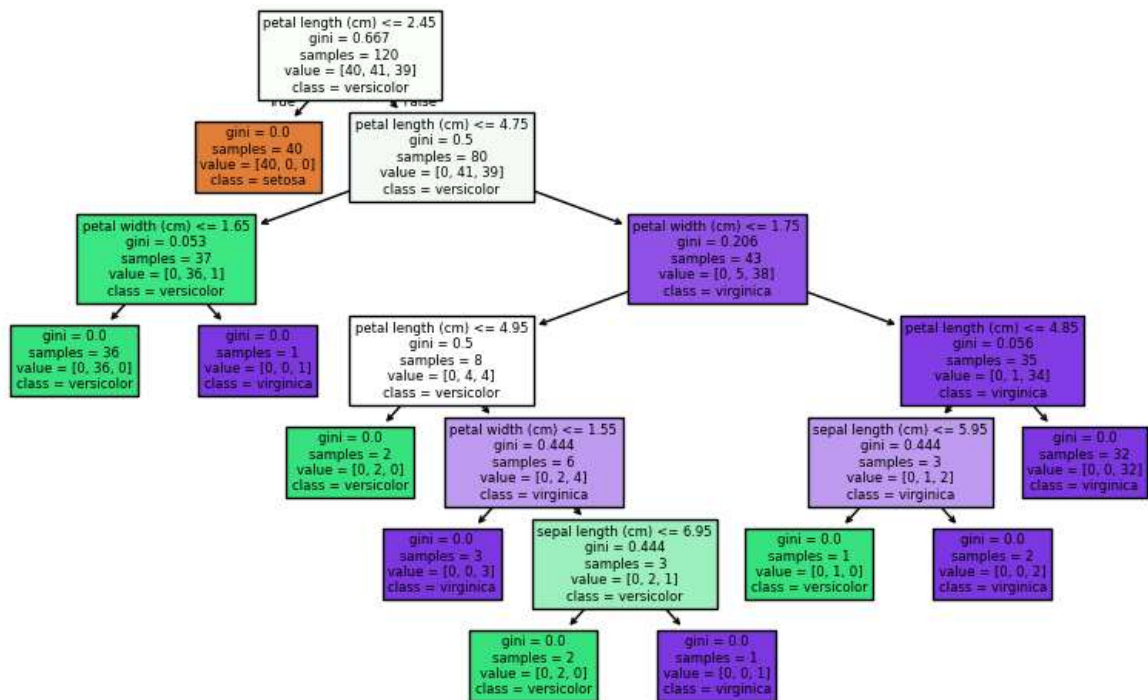
plot_tree(
    model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True
)

plt.title("Decision Tree")
plt.show()

```

Accuracy: 1.0

Decision Tree



```

In [29]: # AdaBoost using Iris Dataset

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.ensemble import AdaBoostClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()

# Create dataframe
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Input and output
X = df[['sepal length (cm)', 'petal length (cm)']]
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create AdaBoost model
model = AdaBoostClassifier(
    n_estimators=50,
    random_state=42
)

# Train model
  
```

```

model.fit(X_train, y_train)

# Predict output
y_pred = model.predict(X_test)

# Accuracy
acc = accuracy_score(y_test, y_pred)

print("Accuracy:", acc)

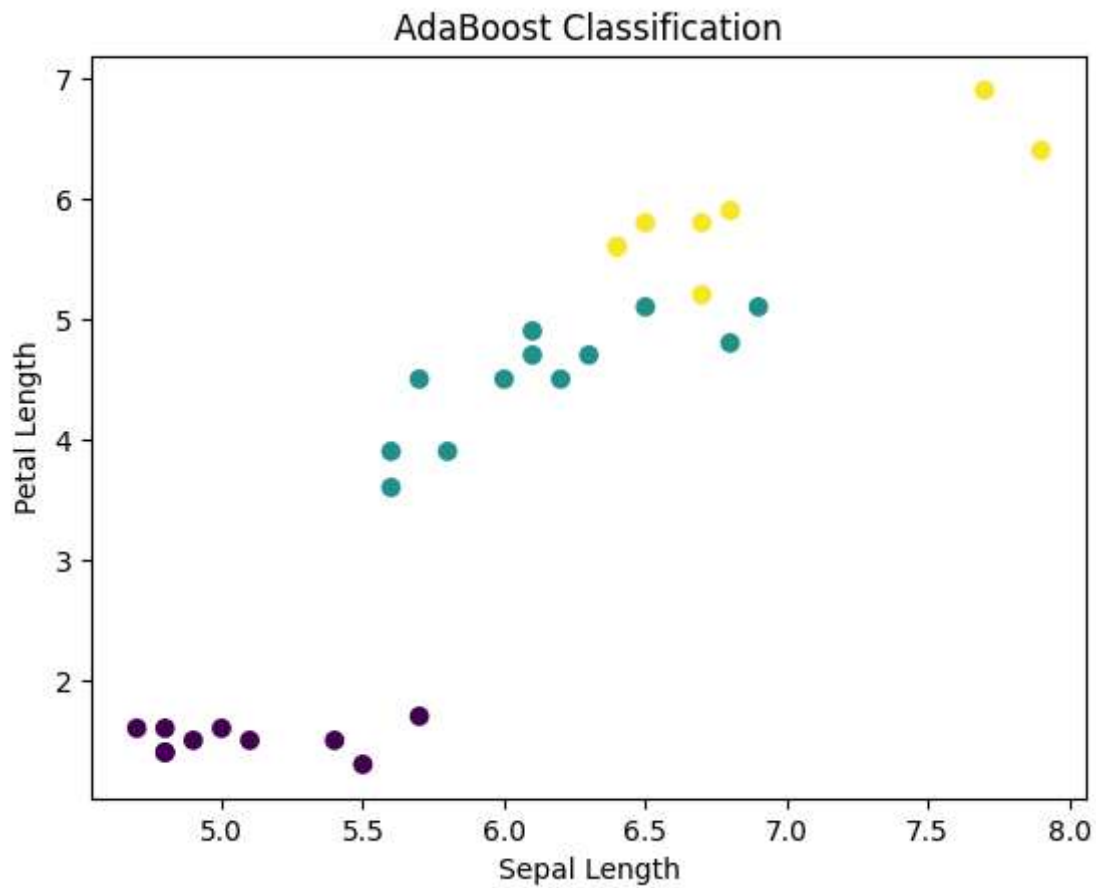
# Plot classification result
plt.scatter(
    X_test['sepal length (cm)'],
    X_test['petal length (cm)'],
    c=y_pred
)

plt.title("AdaBoost Classification")
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")

plt.show()

```

Accuracy: 0.9



```

In [30]: # KNN using Iris Dataset

import pandas as pd
import matplotlib.pyplot as plt

```

```

from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()

# Create dataframe
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Input and output
X = df[['sepal length (cm)', 'petal length (cm)']]
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create KNN model
model = KNeighborsClassifier(n_neighbors=3)

# Train model
model.fit(X_train, y_train)

# Predict output
y_pred = model.predict(X_test)

# Accuracy
acc = accuracy_score(y_test, y_pred)

print("Accuracy:", acc)

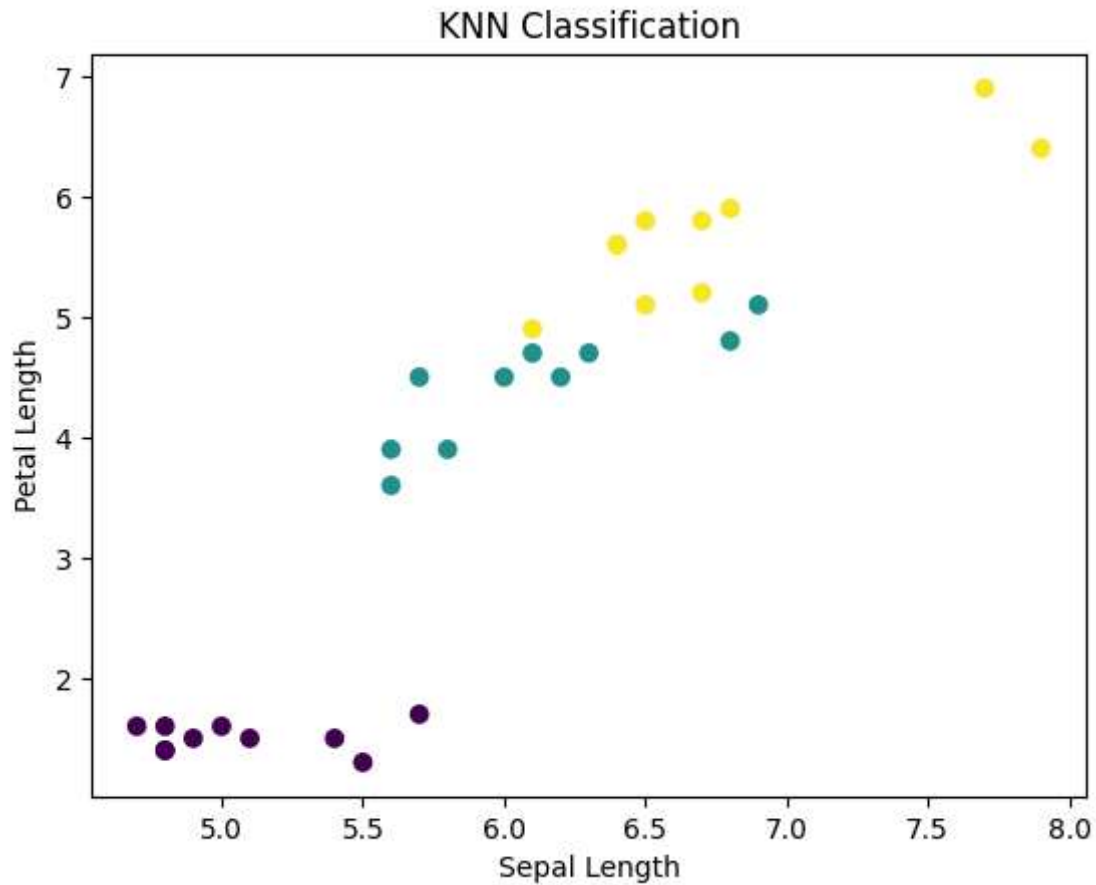
# Plot graph
plt.scatter(
    X_test['sepal length (cm)'],
    X_test['petal length (cm)'],
    c=y_pred
)

plt.title("KNN Classification")
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")

plt.show()

```

Accuracy: 0.9666666666666667



```
In [31]: # K-Means Clustering using Iris Dataset

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.cluster import KMeans

# Load dataset
iris = load_iris()

# Create dataframe
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Input features
X = df[['sepal length (cm)', 'petal length (cm)']]

# Create K-Means model
model = KMeans(
    n_clusters=3,
    random_state=42
)

# Train model
model.fit(X)

# Cluster labels
labels = model.labels_
```

```

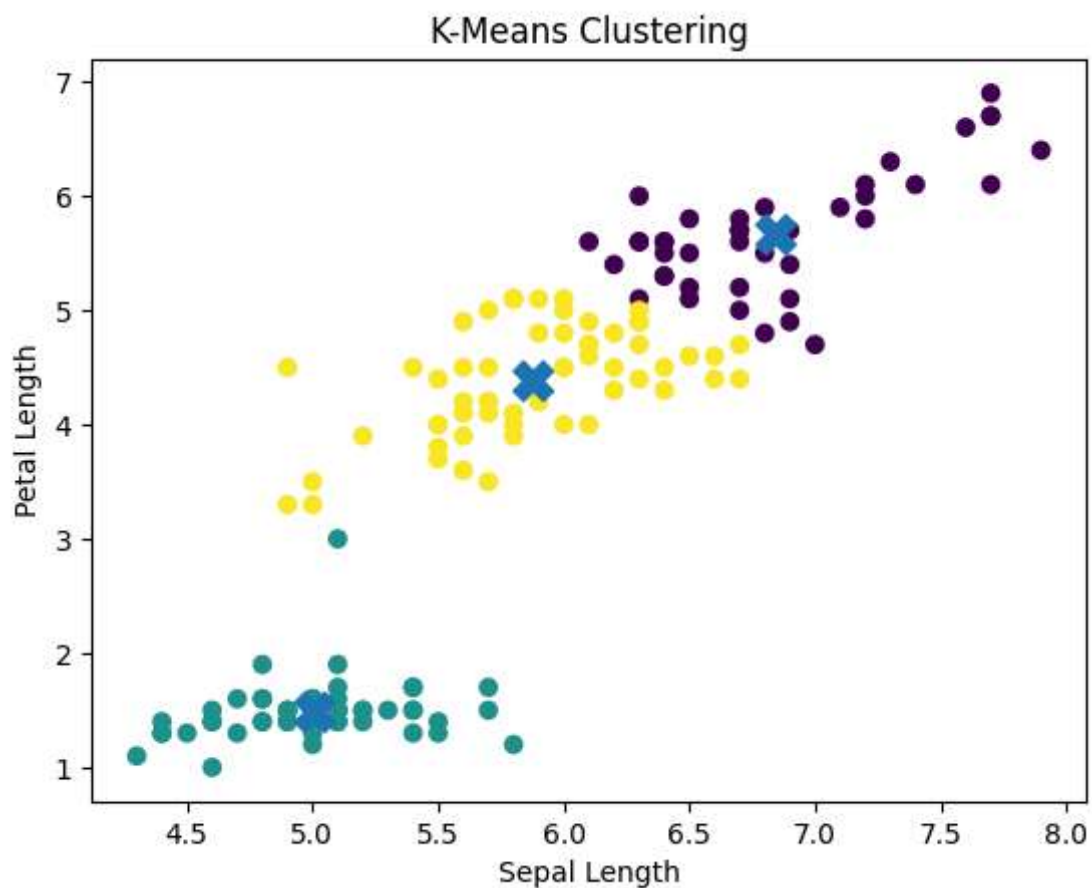
# Plot clusters
plt.scatter(
    X['sepal length (cm)'],
    X['petal length (cm)'],
    c=labels
)

# Plot centroids
plt.scatter(
    model.cluster_centers[:, 0],
    model.cluster_centers[:, 1],
    s=200,
    marker='X'
)

plt.title("K-Means Clustering")
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")

plt.show()

```



```

In [32]: # PCA using Iris Dataset

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris

```



```
from sklearn.decomposition import PCA

# Load dataset
iris = load_iris()

# Create dataframe
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Input data
X = df

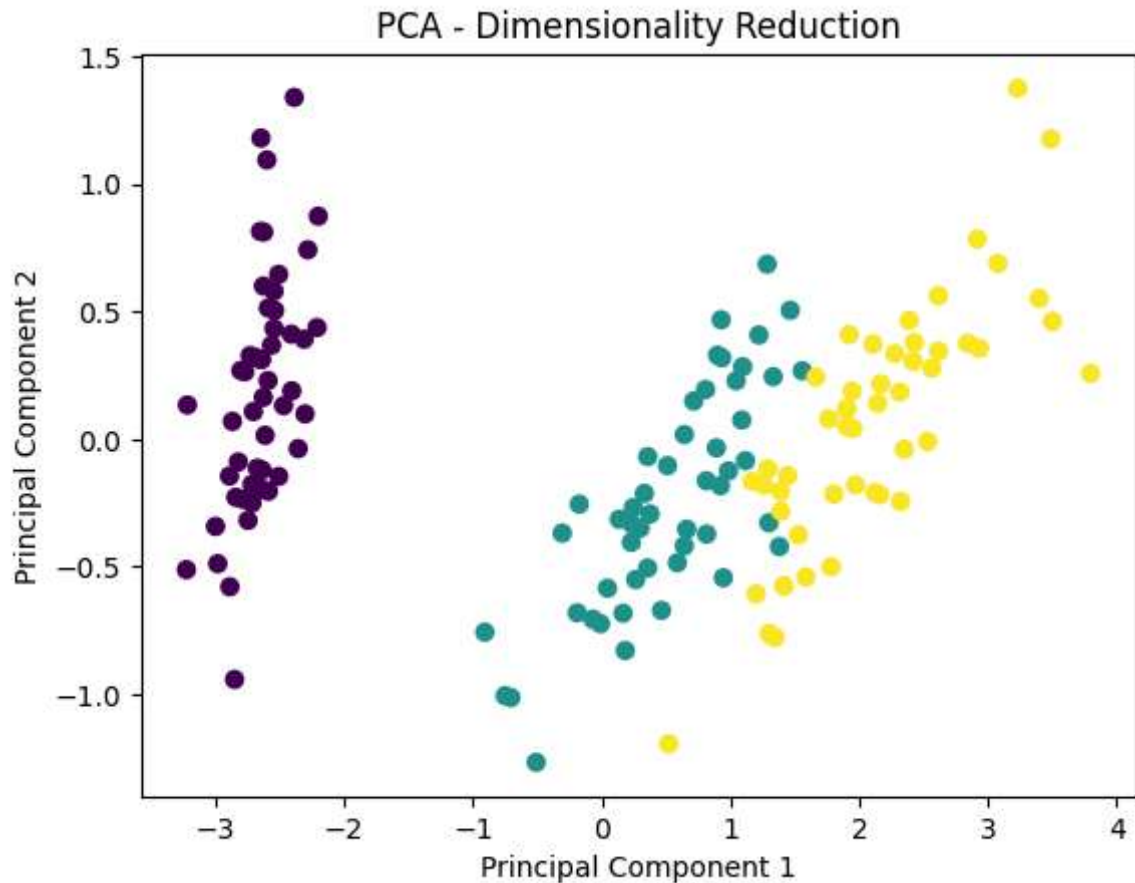
# Create PCA model
pca = PCA(n_components=2)

# Reduce dimensions
X_pca = pca.fit_transform(X)

# Plot reduced data
plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=iris.target
)

plt.title("PCA - Dimensionality Reduction")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.show()
```



```
In [34]: # Simple Linear Regression using Iris Dataset

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.linear_model import LinearRegression

# Load dataset
iris = load_iris()

# Create dataframe
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Input and output
X = df[['sepal length (cm)']]
y = df[['petal length (cm)']]

# Create model
model = LinearRegression()

# Train model
model.fit(X, y)

# Predict values
y_pred = model.predict(X)

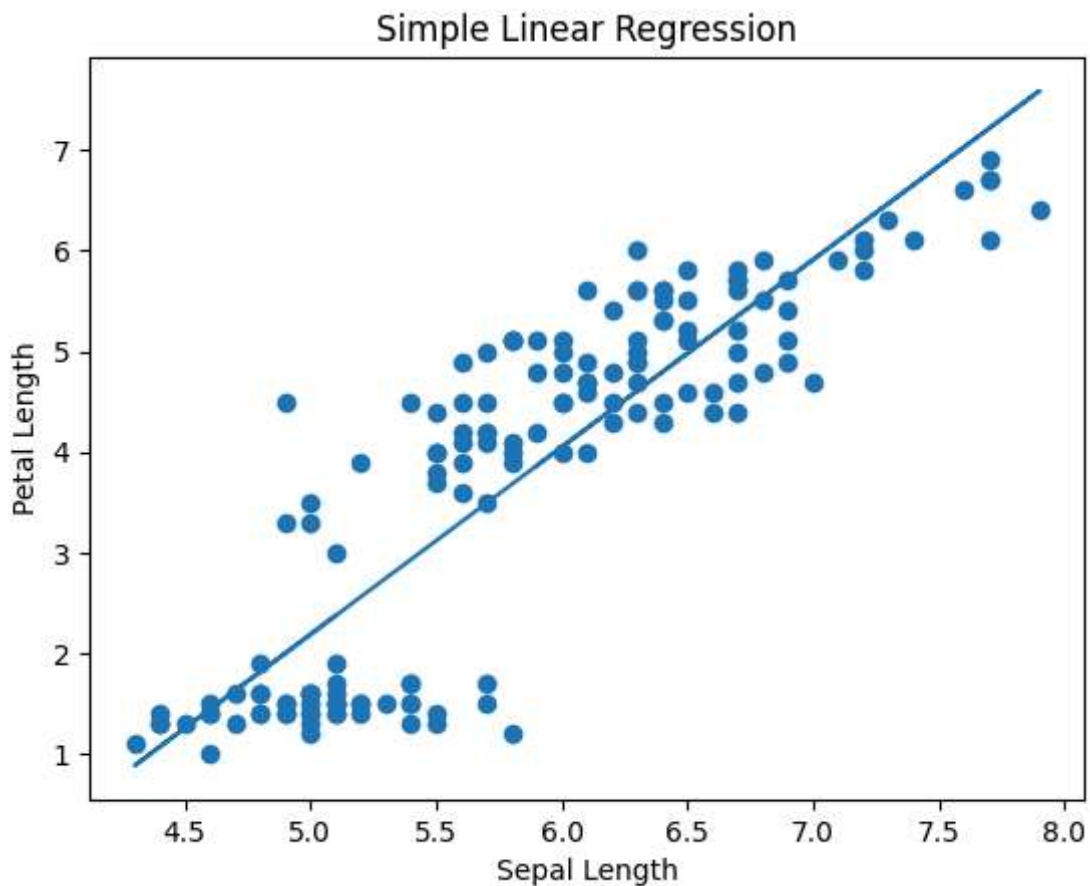
# Plot original points
```

```
plt.scatter(X, y)

# Plot regression line
plt.plot(X, y_pred)

# Labels
plt.title("Simple Linear Regression")
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")

plt.show()
```



```
In [35]: # SVM Classifier using Iris Dataset

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()

# Create dataframe
df = pd.DataFrame(iris.data, columns=iris.feature_names)
```

```

# Input features
X = df[['sepal length (cm)', 'petal length (cm)']]

# Target values
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create SVM model
model = SVC(kernel='linear')

# Train model
model.fit(X_train, y_train)

# Predict output
y_pred = model.predict(X_test)

# Find accuracy
acc = accuracy_score(y_test, y_pred)

print("Accuracy:", acc)

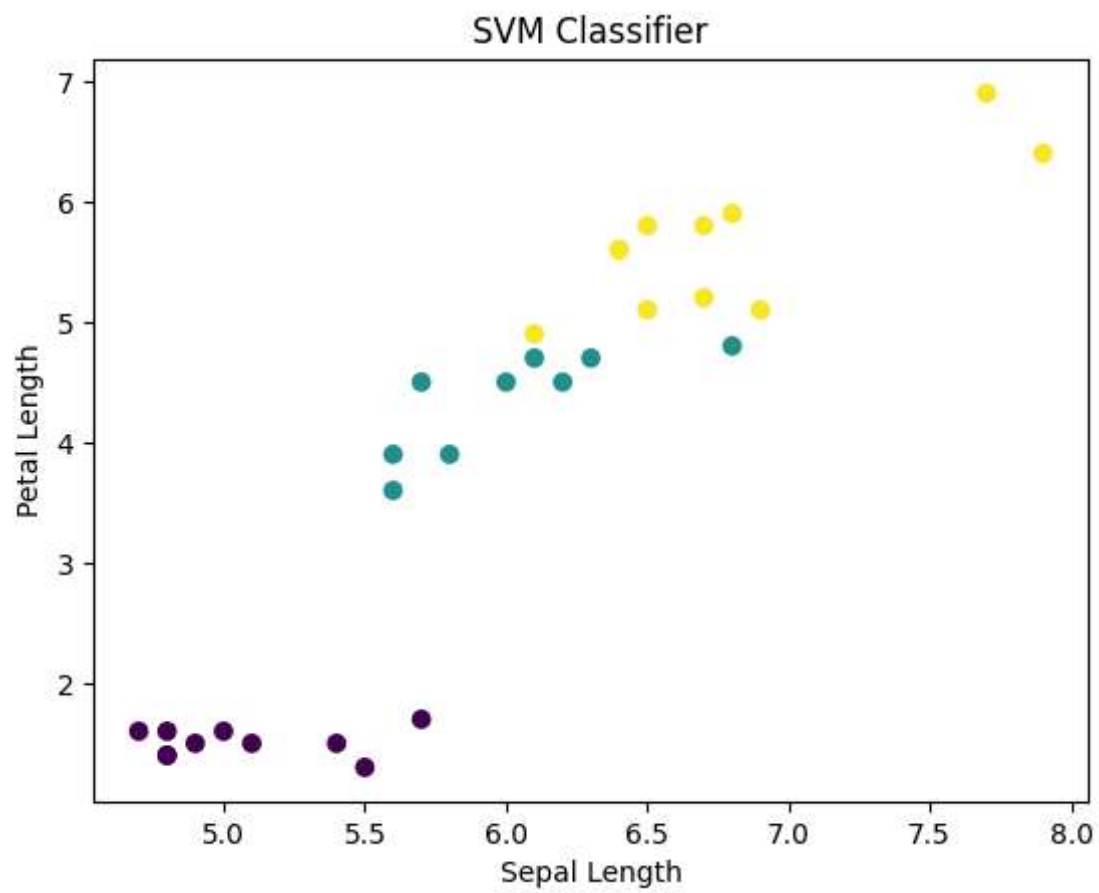
# Plot graph
plt.scatter(
    X_test['sepal length (cm)'],
    X_test['petal length (cm)'],
    c=y_pred
)

plt.title("SVM Classifier")
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")

plt.show()

```

Accuracy: 1.0



In []: